PROJECT REPORT

VIRTUAL AI BOT

A desktop personal AI assistant that can hear, speak, see the screen, and control the computer using voice commands and the Gemini Live API.

Project Title	VIRTUAL AI BOT
Repository	github.com/tanmayikt98-coder/VIRTUAL-AI-BOT

1. Project Purpose

VIRTUAL AI BOT is a desktop personal AI assistant. It can listen to the user through the microphone, speak replies, look at the computer screen or webcam, and perform real actions on the computer such as opening applications, searching the web, managing files, setting reminders, and controlling the browser.

Normal AI chat tools only answer questions in text. They cannot control the user's computer, read what is on the screen, or respond to voice commands. This project tries to solve that limitation by giving a voice-driven assistant that both talks and takes real actions on the machine.

The project was chosen because voice-controlled computer assistants are a practical and interesting application of modern AI. It allows a student to learn real-time audio, computer control, graphical interfaces, and API integration in one system. Target users are students, developers, and everyday users who want a hands-free way to control their PC, search for information, manage files, or get spoken answers without typing. The main objective is to create a cross-platform (Windows, macOS, Linux) voice AI assistant that combines speech input and output, screen awareness, system control, and a visual interface, mainly powered by Google's Gemini Live API.

2. Technologies Used

The following technologies are present in the actual source code and requirements file:

• Python 3.11+ — Main programming language used for the entire application.
• PyQt6 — Builds the graphical user interface, including the holographic avatar, waveform display, settings panels, and video player.
• Google Gemini Live API (google-genai) — Provides real-time two-way voice conversation with

low latency.
• sounddevice and numpy — Capture microphone audio, play spoken replies, and analyse audio for lip-sync.
• Playwright — Controls the web browser (open pages, search, interact with websites).
• mss, Pillow, OpenCV — Capture the screen or webcam and process images for vision features.
• psutil — Monitors system resources such as CPU and RAM usage.
• requests, beautifulsoup4, ddgs — Perform web searches and extract page content.
• FastAPI and uvicorn — Optional remote dashboard so the assistant can be controlled from a phone.
• pyautogui and pyperclip — Control mouse, keyboard, and clipboard.
• yt-dlp — Play YouTube videos inside the interface.

This project does not use HTML, CSS, or JavaScript. It is a native desktop application. No traditional database is used; memory and settings are stored in local JSON files. These technologies were chosen because Python has strong libraries for AI and system control, Gemini Live provides low-latency audio streaming, and PyQt6 gives a modern cross-platform interface without needing a web browser.

3. Working and Logic

Workflow

1. The user starts the application using python main.py or the provided batch file.
2. A setup screen asks for a free Gemini API key.
3. The program opens a continuous Gemini Live audio session.
4. Microphone audio is streamed to Gemini; Gemini’s spoken reply is played back through the speakers.
5. When the model decides an action is needed (open an app, search the web, capture the screen, etc.), it calls a registered tool.
6. The tool runs, returns a clear result (done, unverified, or failed), and the assistant speaks the outcome.
7. A PyQt6 window shows a holographic-style avatar, audio waveform, conversation history, and optional video.

User Input Processing

Voice is captured in real time as PCM audio and sent to Gemini. The user can also type text into the interface. An optional wake-word mode (“Hey Jarvis”) keeps the microphone closed until the phrase is detected.

Decision Making

The Gemini model receives a system prompt that lists every available tool. The tools are auto-discovered at startup from action modules. When the model wants to act, it returns a tool call. The Python side executes the matching function and feeds the result back into the conversation. Dangerous actions such as shutdown or restart are blocked by a confirmation gate until the user presses a button.

Main Components

• main.py — Manages the live session, audio streaming, tool dispatch, echo protection, and wake-word logic.
• ui.py — Builds the graphical interface, renders the avatar, shows settings, and plays video.
• Action modules — Individual skills (web search, file handling, system control, screen capture, etc.) each declare their own tool definition and handler.
• Memory functions — Store and recall user preferences and session summaries.

Output

The system produces spoken replies through the speakers, visual feedback on the HUD (avatar lip-sync, waveform, status messages), and real effects on the computer such as applications opened, browser controlled, files managed, and notifications displayed.

4. Testing and Limitations

No automated test suite is present in the repository. The code contains many try/except blocks and clear error messages. Functionality can only be checked by running the application with a valid Gemini API key and trying voice commands and tool actions.

Challenges Observed

• Real-time audio streaming depends on network quality and API quotas.
• Computer-control features need correct operating-system permissions.
• The published GitHub repository is incomplete (important supporting folders are missing).

Limitations

• Needs an internet connection and a Gemini API key for the default voice engine.
• Many advanced features (local models, full set of actions, plugins) depend on additional packages and folders that are not included in the current repository.
• Performance of vision and browser control varies with hardware and operating system.
• The assistant cannot confirm its own irreversible actions; the user must press a button.

Error Handling

Missing API key, failed tool calls, stalled sessions, and missing optional packages produce clear messages or graceful degradation. The rest of the application continues to run whenever possible.

Possible Future Improvements

• Publish the complete source tree (core/, actions/, memory/, etc.) so the project can actually run from a fresh clone.
• Add a simple smoke-test script.
• Improve the short README so it matches the exact files present in the repository.
• Provide clearer first-run instructions for students who have never used Gemini API keys.

5. GitHub and Originality

Repository Structure

```
VIRTUAL-AI-BOT/
├── .gitignore
├── LICENSE
├── README.md          (very short description)
├── readme.md          (full detailed documentation)
├── main.py            (core engine – large file)
├── ui.py              (PyQt6 interface – large file)
├── requirements.txt
├── setup.py
├── run.bat
└── run_mark_lvi.bat
```

Important note: main.py and ui.py import many packages (core, actions, memory, etc.). These folders are not present in the published repository. A fresh clone therefore cannot run without obtaining the missing source code. The detailed documentation and code comments show that the software is based on Mark LVI, an open-source personal AI assistant created by FatihMakes.

Execution Steps

When the full source is available:
1. Install Python 3.11 or 3.12.
2. Run python setup.py to install dependencies and Playwright browsers.
3. Launch with python main.py or the provided .bat file.
4. Enter a free Gemini API key when the setup screen appears.

Individual Contributions

I worked on understanding the overall architecture, studying the voice session flow, the tool system, and the user interface. I also organised and presented the project for academic submission. The core large codebase originates from the Mark LVI project.

AI Assistance

AI tools were used for coding assistance, understanding the architecture, and preparing documentation.

Learning Outcomes

• How a real-time voice AI session is structured.
• How a large language model can call local computer-control tools.
• Building a desktop GUI with PyQt6.
• The importance of publishing a complete and runnable source repository.
• The value of clear attribution when using open-source code.

Conclusion

VIRTUAL AI BOT is an ambitious personal AI assistant that can listen, speak, see the screen, and control the computer. The main ideas—Gemini Live audio, tool calling, PyQt6 interface, and modular actions—are clearly visible in the source files. However, the published repository is incomplete (supporting packages are missing) and is derived from the open-source Mark LVI project by FatihMakes. With the full source restored and proper credit given, the project remains a rich learning platform for students interested in voice AI and desktop automation.

— End of Report —